

CS 113

Week 5 Lecture 1



Today's Topics

Homework #3

Quiz #1 next week

Useful String methods

Convenience operators

Recursion vs Loops (2 final examples)

Arrays

Homework #3

Homework #3: will be due next Tues Oct 6.

Quiz #1

Quiz in lab next Tuesday.

Topics covered will be announced asap.

Useful String Methods

length

compareTo

contains

charAt,

substring

split

Recursion

Recursion: When a method calls itself to solve a smaller version of the same problem.

A recursive method needs:

1. **Base case:** when do we stop?
2. **Recursive case:** how do we make the problem smaller?

Loops

- Easy way to repeat some computation
- Two kinds of loops:
 - While
 - For
- Loops repeat block of code until the condition becomes false

Accumulator pattern

Idea: Repeatedly update a variable (typically in a loop)

Pattern:

1. Initialize accumulator variable
2. Loop until done
 1. Update the accumulator variable

Loops vs Recursion: Palindromes

Loops vs Recursion: Fibonacci

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

`sum = sum + 2`

`count = count + 1`

`count = count - 1`

`product = product * 2`

`divisor = divisor / 2`

`message = message + "lol!"`

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

sum = sum + 2	
count = count + 1	
count = count - 1	
product = product * 2	
divisor = divisor / 2	
message = message + " lol"	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

sum = sum + 2	sum += 2
count = count + 1	
count = count - 1	
product = product * 2	
divisor = divisor / 2	
message = message + " lol"	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	
<code>product = product * 2</code>	
<code>divisor = divisor / 2</code>	
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	
<code>divisor = divisor / 2</code>	
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	<code>product *= 2</code>
<code>divisor = divisor / 2</code>	
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	<code>product *= 2</code>
<code>divisor = divisor / 2</code>	<code>divisor /= 2</code>
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	<code>product *= 2</code>
<code>divisor = divisor / 2</code>	<code>divisor /= 2</code>
<code>message = message + " lol"</code>	<code>message += " lol"</code>

Example: Palindrome

Loop vs recursion

Arrays

If I want to store 5 students grades, I need 5 different variables.

```
int grade1 = 90;  
int grade2 = 85;  
int grade3 = 92;  
...
```

Solution: Arrays

- One variable name
- Many values of the same type
- **Cannot change size**

```
int[] grades = {90, 85, 92, 78, 88};
```

Arrays

Indices start at 0!

length() gets number of elements in the array

- Don't Java strings have this function too?...

```
int[] grades = {90, 85, 92, 78, 88};
```

```
int numGrades = grades.length(); // will return 5
```

Reading / Writing Array Elements

```
int[] grades = {90, 85, 92, 78, 88};
```

```
System.out.println("Second grade in list is: " + grades[1]);
```

```
grades[2] = 70;
```

```
int[] grades = {90, 85, 70, 78, 88};
```

Out of bounds example

```
int[] grades = {90, 85, 92, 78, 88};
```

```
System.out.println("Last grade in list is: " + grades[5]);
```

```
ArrayIndexOutOfBoundsException!
```